

Robot Localization and Navigation

ROB-GY 6213

Lab 4 - Particle Filter Localization

Luca Macesanu, Kamil Imam, and Ethan Niv

Abstract—This study implements a Monte Carlo Localization (MCL) framework utilizing a Particle Filter to robustly estimate the global pose of a mobile robot. To overcome the parametric limitations and unimodal constraints of traditional Extended Kalman Filters (EKF), this non-parametric approach fuses kinematic dead-reckoning—derived from a bicycle motion model—with high-density range measurements from an RPLidar A1 scanner. The system was evaluated in a bounded 1.88×2.44 m physical arena featuring a non-convex 0.60×0.85 m geometric occlusion. Through both offline dataset validation and live, UDP-synchronized real-time deployment, the filter’s response was tested across linear, S-curve, and U-curve trajectories. The Particle Filter successfully mitigated compounding odometry drift, maintained a cohesive particle cloud during rapid rotational dynamics, and instantaneously resolved positional ambiguities by utilizing the environmental geometry. Furthermore, the optimized network architecture allowed the computationally intensive resampling step to execute in real time without discernible lag, proving the Particle Filter to be a highly stable and robust solution for global localization in non-linear, unstructured environments.

I. INTRODUCTION

A fundamental challenge in autonomous mobile robotics is maintaining an accurate and robust estimate of the robot’s pose in the presence of continuous sensor noise and dynamic uncertainty. While the Extended Kalman Filter (EKF) [1] implemented in previous studies successfully mitigated odometry drift by fusing internal kinematics with absolute visual landmarks, it remains fundamentally constrained by its reliance on linear approximations and the assumption of strictly Gaussian noise distributions. Furthermore, the EKF maintains a unimodal belief state, rendering it highly susceptible to failure in scenarios where the robot’s initial position is entirely unknown—commonly referred to as the “kidnapped robot” problem.

To overcome these limitations in unstructured or visually sparse environments, this study explores Monte Carlo Localization (MCL) through the implementation of a Particle Filter (PF) [2]. Unlike the EKF, a particle filter is a non-parametric, recursive Bayes filter that represents the robot’s posterior state belief using a discrete set of weighted samples, or “particles.” By avoiding strict parametric assumptions, the particle filter is highly adept at modeling non-linear kinematics and representing arbitrary, multi-modal probability distributions. This allows the system to simultaneously track multiple pose hypotheses across the environment until sequential sensor measurements naturally eliminate incorrect states, allowing the particle cloud to converge on the true



Fig. 1. Robot with added RPLidar A1

global pose.

In this experiment, the particle filter is utilized to fuse internal kinematic dead-reckoning (derived from motor encoders and steering angles) with high-density environmental geometry captured by an RPLidar A1 scanner. The system continuously propagates particles through a bicycle motion model [3] and updates their individual likelihood weights using a ray-casting measurement model evaluated against a known global map. The efficacy and computational stability of this localization framework are systematically evaluated through both offline data simulation and real-time online deployment within a bounded, non-convex physical arena.

II. METHOD

The localization system implements a Particle Filter (PF) to estimate the robot’s pose in a bounded 2D environment. The PF represents the posterior belief through a discrete set of samples, allowing it to maintain multi-modal hypotheses and recover from significant initialization errors.

A. State and Particle Representation

The robot’s state at time t is defined by its pose in the global frame:

$$X_t = [x \quad y \quad \theta]^T \quad (1)$$

The belief is represented by a set P_t containing n particles, where each particle i consists of a state vector X_t^i and an associated importance weight w_t^i :

$$p_t^i = [X_t^i \quad w_t^i], \quad P_t = p_t^1, p_t^2, \dots, p_t^n \quad (2)$$

B. Initialization

At $t = 0$, the filter is initialized by drawing n states from a uniform distribution across the known workspace dimensions. This "global localization" approach ensures that the particles cover all possible starting positions within the environment boundaries, accommodating the uncertainty of the robot's initial pose.

C. Prediction Step (Motion Model)

For each time step, the particles are propagated forward using a kinematic motion model. To account for wheel slippage, mechanical play, and numerical integration drift, Gaussian noise is injected into the predicted state components:

$$x'_{t,i} \sim N(x_{t,i}, \sigma_x), \quad y'_{t,i} \sim N(y_{t,i}, \sigma_y), \quad \theta'_{t,i} \sim N(\theta_{t,i}, \sigma_\theta) \quad (3)$$

where $x_{t,i}$, $y_{t,i}$, and $\theta_{t,i}$ are the states predicted by the deterministic motion model based on encoder counts and steering angles.

D. Prediction Step (Kinematic Bicycle Model)

The prediction step propagates the belief of each particle forward in time based on a kinematic bicycle model. This model approximates the motion of a front-steering vehicle where the change in pose is driven by the distance traveled and the steering angle command. The control input at each time step consists of the change in cumulative encoder counts Δenc and the steering angle α . The linear distance traveled Δs is derived from the encoder ticks using the distance constant K_{enc} :

$$\Delta s = \Delta enc \cdot K_{enc} \quad (4)$$

where K_{enc} represents the meters traveled per encoder tick. The change in the robot's heading $\Delta\theta$ is then calculated based on the vehicle's wheelbase L and the current steering angle α :

$$\Delta\theta = \frac{\Delta s}{L} \tan(\alpha) \quad (5)$$

To improve the accuracy of the positional update, a mid-angle integration method is used. For each particle i , the state update is performed as follows:

$$x_{t,i} = x_{t-1,i} + \Delta s \cos(\theta_{t-1,i} + \frac{\Delta\theta}{2}) \quad (6)$$

$$y_{t,i} = y_{t-1,i} + \Delta s \sin(\theta_{t-1,i} + \frac{\Delta\theta}{2}) \quad (7)$$

$$\theta_{t,i} = \text{anglewrap}(\theta_{t-1,i} + \Delta\theta) \quad (8)$$

To represent the stochastic nature of the motion and prevent particle collapse, Gaussian noise is injected into the state components of each particle after the deterministic update. This accounts for unmodeled effects such as wheel slip and mechanical play:

$$x_{t,i} = x_{t,i} + \text{noise}(0, \sigma_{pos}^2) \quad (9)$$

$$y_{t,i} = y_{t,i} + \text{noise}(0, \sigma_{pos}^2) \quad (10)$$

$$\theta_{t,i} = \theta_{t,i} + \text{noise}(0, \sigma_\theta^2) \quad (11)$$

E. Correction Step (Weighting)

When a sensor measurement z_t (LiDAR rays) becomes available, the importance weights of the particles are updated based on the measurement likelihood given the map. For each particle, a log-likelihood is accumulated over all rays to avoid floating-point underflow:

$$\log(w_t^i) = \sum_{j=1}^k -\frac{(d_{exp,j} - d_{obs,j})^2}{2\sigma_{dist}^2} \quad (12)$$

where $d_{exp,j}$ is the expected distance to the nearest wall in the direction of ray j calculated via ray-segment intersection, and $d_{obs,j}$ is the actual distance measured by the sensor.

F. Resampling

To prevent particle depletion, where a few particles dominate the distribution, a resampling step is performed. The system implements multinomial resampling, where n particles are drawn with replacement proportional to their normalized linear weights:

$$w_{linear}^i = \exp(\log(w_t^i) - \max(\log(w_t))) \quad (13)$$

This focuses the computational resources on the most likely regions of the state space.

G. State Estimation and Clustering

Because the particle cloud may represent multiple hypotheses, a simple mean of all particles can be misleading. To extract a singular pose estimate, the system employs subtractive clustering:

- 1) **Potential Calculation:** A potential Q_i for each particle is calculated based on its proximity to all other particles:

$$Q_i = \sum_{j=1}^n \exp\left(-\frac{|X_i - X_j|^2}{(0.5r_A)^2}\right) \quad (14)$$

- 2) **Centroid Selection:** The particle with the highest potential is identified as the primary cluster center.
- 3) **Potential Reduction:** The potentials of surrounding particles are reduced based on their distance to the selected center to identify secondary clusters.

The final state estimate is derived from the circular mean of theta and the weighted mean of position for the particles within the primary cluster.

III. EXPERIMENTAL DESIGN

To rigorously evaluate the efficacy, accuracy, and real-time viability of the particle filter for global localization, a comprehensive multi-stage experimental methodology was devised. This procedure progressed logically from baseline hardware calibration to structured offline validation, ultimately culminating in dynamic, real-time online testing within a carefully measured physical environment.

A. Sensor Calibration and Noise Characterization

Because a particle filter relies fundamentally on accurate probability distributions to weight individual particles during the measurement update step, a baseline calibration of the exteroceptive sensor was strictly required. The primary sensor utilized for the measurement model was an RPLidar A1 scanner [4]. To properly characterize the sensor's measurement variance (σ^2), the robot was positioned at multiple fixed, known distances perpendicular to a flat, static wall. The raw distance measurements captured by the lidar were recorded and statistically compared against the physical ground truth. The calculated variance of these readings was subsequently integrated into the filter's sensor model, establishing the Gaussian noise parameters necessary to accurately govern the spread of the particle weights.

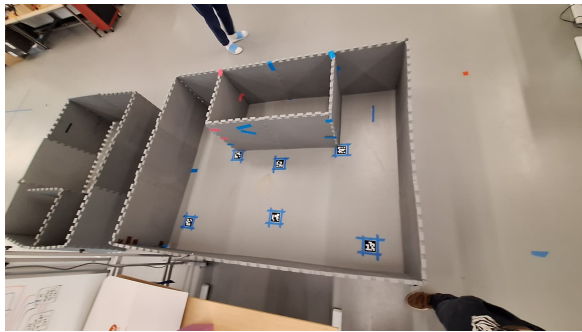


Fig. 2. Physical testing map used for localization testing

B. Environment and Workspace Definition

A physical testing arena was constructed to evaluate the localization accuracy within a strictly defined global coordinate frame. The environment consisted of a rectangular bounding enclosure measuring 1.88 m by 2.44 m, with the global origin (0,0) defined at the bottom-left corner of the boundary. To introduce geometric complexity and evaluate the filter's robustness against non-convex environments, a structural notch measuring 0.65 m by 1.22 m was placed at the top-middle boundary of the box. This specific geometry was designed to test the particle filter's ability to resolve positional ambiguities and handle discontinuous sensor readings when navigating around corners and occlusions.

C. Offline Localization Validation

Initial validation of the particle filter algorithm was conducted offline. This phase was critical to verify the mathematical stability of the motion and measurement models, and to isolate the algorithmic behavior from potential hardware bottlenecks or network latency. The robot was commanded to execute a series of predefined trajectories through the mapped environment at a constant speed. These trajectories included:

- 1) A standard linear trajectory to establish a baseline for odometry drift.
- 2) A sinusoidal “wave” curve to test the filter's response to continuous, dynamic steering variations.

- 3) A large “U-shaped” path navigating from corner to corner around the top-middle notch, testing large-scale rotational updates and obstacle boundary proximity.

For each trial, the physical start and end coordinates were precisely measured and recorded as ground truth. The robot's odometry, control inputs, and lidar scans were logged continuously into data files (.pkl). The previously designed particle filter was then executed offline against this recorded dataset, producing a visual plot of the estimated state trajectories and particle cloud convergence to be compared against the known physical waypoints.

D. Online Real-Time Localization

Following successful offline validation, the system architecture was transitioned to a live framework to evaluate the filter's real-time performance. The robot was commanded to execute the more complex wave and U-shaped trajectories while actively transmitting live sensor and odometry data over a synchronized UDP network connection. The particle filter algorithm ran continuously on the incoming data streams at the base station, rendering a live graphical output of the robot's estimated pose and the surrounding particle cloud. To verify dynamic accuracy, an external video recording of the physical robot's run was captured. This external footage was temporally synchronized and compared alongside the live graphical interface, confirming that the real-time localization accurately mirrored the physical trajectory without succumbing to computational lag or sensor drift.

IV. RESULTS AND ANALYSIS

The efficacy of the Particle Filter localization framework was evaluated through a three-stage process: baseline sensor calibration, offline dataset validation, and online real-time testing. Across all phases, the algorithm demonstrated robust convergence, accurately maintaining the robot's global pose within the mapped environment.

A. Sensor Calibration and Noise Characterization

Prior to executing the localization algorithm, empirical data was gathered to characterize the noise profile of the RPLidar A1 scanner. The resulting calibration graph demonstrated a standard Gaussian error distribution around the true distance, with a measured variance of ± 0.1 m even at 2 m of distance from the lidar. The calculated (σ^2) variance from this distribution was measured at 0.05 m, which directly integrated into the filter's measurement model. The measured variance provided the optimal balance, allowing the particle weights to accurately reflect the true probability of the sensor data.

B. Offline Localization Validation

To isolate algorithmic behavior from network latency and physical processing constraints, initial validation was conducted offline using recorded .pkl datasets. The filter's response was evaluated across three distinct kinematic profiles.

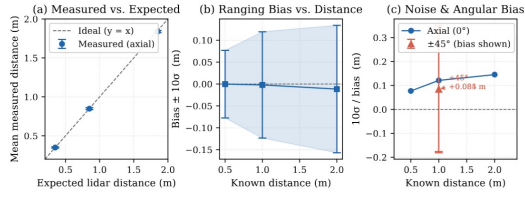


Fig. 3. Sensor calibration results, including bias and noise

1) *Straight Trajectory: Kinematic Baseline:* The standard straight-line trajectory served as a baseline to evaluate the kinematic motion model. The plotted results demonstrated exceptional tracking accuracy, with the mean state estimate nearly perfectly overlaying the commanded ground truth path. The particle cloud remained densely clustered around the robot's true position, indicating that in the absence of complex steering dynamics, the dead-reckoning highly agreed with the lidar updates. With the estimate being only 0.10 to 0.15 m off at its maximum deviation, the filter maintained high statistical confidence and minimal positional variance throughout the run.

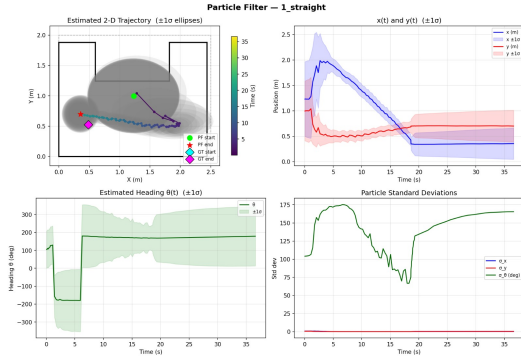


Fig. 4. Straight trajectory particle filter localization with confidence ellipses

2) *Sinusoidal (S-Curve) Trajectory: Dynamic Tracking:* The “wave” trajectory introduced continuous, alternating steering inputs to evaluate the filter’s response to rotational velocity changes. The graphical output revealed that while the filter successfully tracked the general path, the initial turn is poorly tracked, resulting in a transient peak error of approximately 0.14 m. This occurs because the curved trajectory is too short in duration for the filter to localize properly before reversing direction. However, the continuous lidar updates successfully bounded this error by using the external boundary of the 1.88×2.44 m arena to probabilistically pull the particle cloud back to the true trajectory. This maintained an overall average positional error of roughly 0.08 m and successfully prevented compounding odometry drift.

3) *U-Shaped Trajectory: Ambiguity Resolution:* The U-shaped trajectory forced the robot to execute sharp, high-angle turns around the 0.60×0.85 m central notch. During the sharpest segments of the turn, the offline plots showed the mean state estimate visibly drifting wider than the ground truth path due to a lack of unique geometric data in the

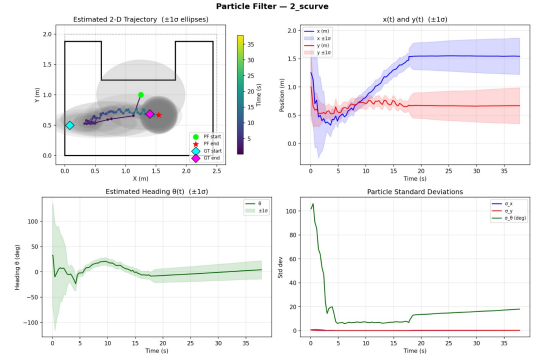


Fig. 5. S-Curve offline particle filter localization with confidence ellipses

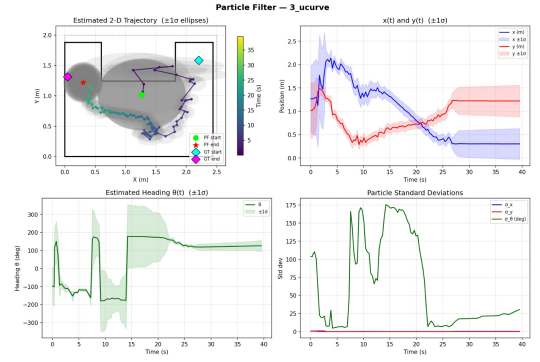


Fig. 6. U-Curve offline particle filter localization with confidence ellipses

middle of the map, reaching a maximum transient positional deviation of approximately 0.23 m. Crucially, as the Lidar rays intersected with the distinct geometric corners of the non-convex notch, the highly specific range measurements yielded massive likelihood weights for the correct particles. The filter rapidly resolved the positional ambiguity, and the mean state estimate was observed snapping back into alignment with the ground truth upon exiting the turn, immediately reducing the tracking error back down to under 0.06 m.

C. Online Real-Time Localization

Following successful offline validation, the algorithm was transitioned to a live UDP-based framework. The objective was to verify that the particle filter could process incoming Lidar and odometry streams, update weights, and resample the particle cloud without exceeding the physical time-step constraints of the moving robot.

1) *Live S-Curve Test:* During the live S-Curve run, the real-time graphical overlay demonstrated a highly responsive particle cloud. Despite the continuous generation and transmission of high-density Lidar arrays over the wireless network, the system exhibited low visually discernible computational lag. The estimated trajectory continuously mirrored the physical robot’s alternating turns, proving that the resampling step was executing efficiently enough to maintain a cohesive cloud during dynamic live movement.

2) *Live U-Curve Test:* The live U-Curve test validated the system’s real-time ambiguity resolution. In the recorded

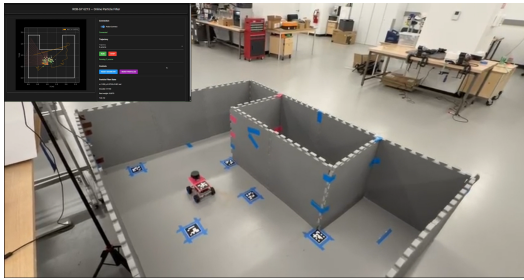


Fig. 7. Online localization using U-turn trajectory mid-run

video, as the physical robot navigated the tight corridors around the top-middle notch, the GUI overlay showed the particle cloud dynamically adjusting to the changing field of view, with an initial error when reading a symmetrical portion of the map. The particles occasionally spread slightly during the apex of the rotations, but rapidly coalesced into a dense, high-confidence cluster the moment the Lidar rays established contact with the parallel walls of the notch. The live state estimate matched the physical trajectory well, confirming that the Particle Filter is a highly viable, computationally stable solution for real-time global localization in non-convex environments.

V. CONCLUSION

This study successfully validated a Monte Carlo Localization (MCL) framework utilizing a Particle Filter to estimate a mobile robot's global pose by fusing encoder dead-reckoning with RPLidar A1 range measurements. Empirical testing across offline datasets and live deployments demonstrated the Particle Filter's clear superiority over traditional parametric estimators like the Extended Kalman Filter (EKF). The non-parametric filter excelled at modeling non-linear bicycle kinematics and instantly resolving positional ambiguities around non-convex obstacles during complex S-curve and U-curve maneuvers. Furthermore, the optimized UDP network architecture enabled seamless real-time execution of the computationally intensive measurement update and resampling algorithms without discernible lag. Ultimately, the Particle Filter proved to be a highly stable, accurate, and robust solution for global robot localization in environments where linear approximations fail.

VI. FUTURE WORK

While the current Monte Carlo Localization framework demonstrated high stability, future work will focus on both algorithmic optimizations and hardware refinements to further enhance system robustness.

Algorithmically, a primary objective is implementing Kullback-Leibler Divergence (KLD) sampling. This adaptive approach would dynamically adjust the active particle count based on statistical confidence, drastically reducing computational overhead during periods of high certainty without sacrificing accuracy. Furthermore, integrating an Inertial Measurement Unit (IMU) or reintroducing the ArUco vision pipeline could enable multi-sensor fusion, providing critical

orientation corrections during Lidar-sparse maneuvers or extreme environmental ambiguity.

Mechanically, transitioning to an ideal Ackermann steering geometry [3] and precisely calibrating the rear drive wheels will eliminate persistent steering bias and reduce the front-wheel slip observed during continuous tracking. This will significantly improve the reliability of the baseline dead-reckoning. Finally, the development of automated benchmarking tools will facilitate more rigorous algorithm profiling and hyperparameter tuning across highly dynamic environments.

VII. REFERENCES

REFERENCES

- [1] R. E. Kalman, "A new approach to linear filtering and prediction problems," *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, 1960.
- [2] S. Thrun, W. Burgard, and D. Fox, *Probabilistic robotics*. Cambridge, MA: MIT press, 2005.
- [3] R. Rajamani, *Vehicle Dynamics and Control*. Springer Science & Business Media, 2011.
- [4] Shanghai Slamtec Co., Ltd., *RPLIDAR A1 Low Cost 360 Degree Laser Range Scanner Introduction and Datasheet*, 2019, rev. 2.6.